

Recipes infer: 推理组件升级，开箱即用

分享嘉宾：田佳渊
2026.06.27 中国·上海



 CANN Recipes 概述

 统一模型使用，加速模型适配

 推理组件实现细节

 总结与未来计划

CANN Recipes 概述

推理 / 训练 / 具身智能 / 模型量化

<https://gitcode.com/cann/cann-recipes-infer>

基于CANN平台的开源推理仓库

模型实践

训练

推理

模型量化

具身智能

鸿蒙推理

[查看更多 >](#)

DeepSeek-V4系列推理优化样例

支持DeepSeek-V4在昇腾Atlas A3 Pod平台和950PR/DT平台部署，提供长...

 deepseek-v4

Qwen3.5推理优化样例

在昇腾Atlas A3 Pod平台上完成Qwen3.5-35B-A3B模型基于框架的快速部署...

 qwen3.5

HunyuanVideo推理优化样例

支持HunyuanVideo在昇腾NPU环境部署，面向720p长序列场景提供高性能...

 HunyuanVideo

Wan2.2-I2V推理优化样例

支持Wan2.2-I2V在昇腾NPU环境部署，提供基于图像输入的高性能视频生成...

 Wan2.2-I2V

Cann-recipes-infer 提供了LLM 与多模态热点模型的优化样例，LLM 涵盖 DeepSeek 系列、Qwen 系列、Kimi 系列、LongCat 系列、GLM 5、Gpt-oss、Gemma-4等多个模型样例。

统一模型使用，加速模型适配

对比项	升级前：独立的轻量级工程脚本	升级后：统一推理组件 + 独立模型结构
执行入口、环境版本	多入口： 各模型自定义执行入口 infer.sh，执行不同模型需要拉起不同的入口 多版本： 各模型支持的环境版本不统一，切换模型还需切换环境	单入口： 所有模型拉起脚本统一调用 executor/sctrips/infer.sh 单版本： 所有模型使用统一的CANN/torch_npu/transformers版本
公共流程、基础能力	重复开发： 各模型自行处理	无需开发： 由公共推理组件管理
适配体验	适配成本高： 读脚本、抄流程、逐项调试全链路	适配简化： 注册模型，重点实现模型内部的优化即可
量化精度评测	功能缺失： Infer仓内无定量测评精度工具	新增功能： 在 infer 仓内即可通过 在线数据集评测 完成量化前后精度对比

适配成本高

新模型接入需要打通端到端推理闭环
需要从 prompt 输入到结果返回完整打通，链路长且学习成本高，相同的功能需要重复开发。

精度难验证

优化效果缺少定量判断依据
量化、融合算子等优化完成后，缺少定量测评指标，难以判断是否引入精度偏差。

推理组件升级

适配成本变低

新模型只关注优化模型本体
节省“打通推理流程”的工作，开发者只需专注于“优化模型本体”。
代码量平均减少 600+。

精度变化可感知

新增在线数据集测评
在线模式支持数据集测评，对量化、图编译等优化项，可形成“优化前后可对照”的验证闭环。

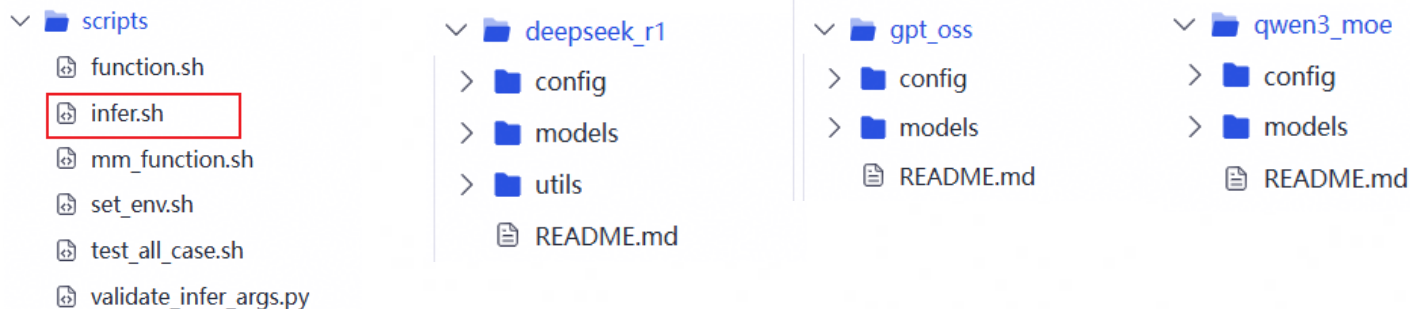
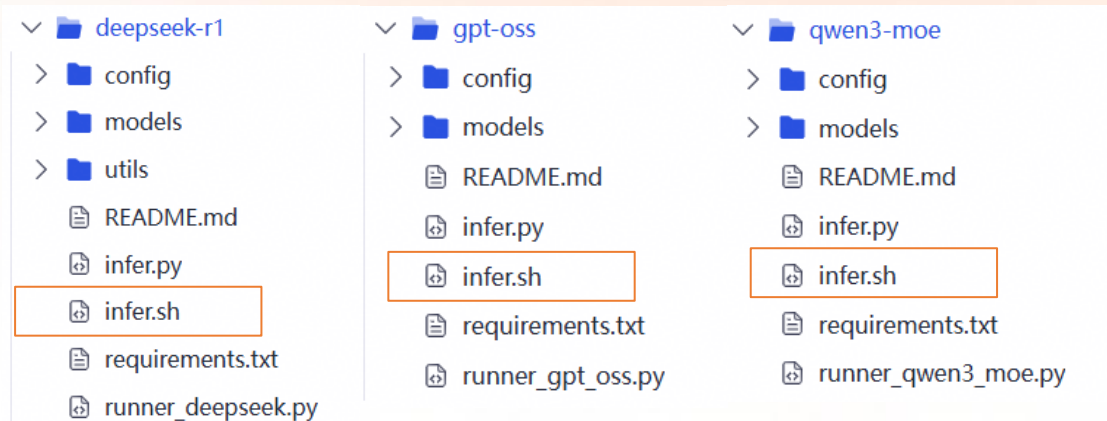
统一模型使用，加速模型适配

运行模型推理：入口统一

多入口  单入口

多个模型接入统一的执行脚本，只需更改配置参数即可一键拉起：

```
bash executor/scripts/infer.sh \  
  --model <model> --yaml <yaml>
```



统一模型使用，加速模型适配

运行模型推理：环境统一

多版本  单版本

升级后将主要模型收敛到**统一环境组合**，用户运行不同模型时无需反复切换 CANN / PyTorch / torch_npu / transformers 版本。

升级前：模型各自声明环境

CANN: 8.3.RC1.alpha002 / 8.3.RC1.alpha003 / 8.5.0.alpha002 /
镜像 cann8.5 / cann8.3.rc2
torch_npu: / PyTorch: v7.3.0 / 2.6.0 为主
transformers: 4.53.3 / 4.55.0

升级后：所有模型统一环境

CANN: 统一到 CANN社区最新版本
torch_npu / PyTorch: CANN社区最新版本 / 2.8.0
transformers: 5.0.0

统一模型使用，加速模型适配

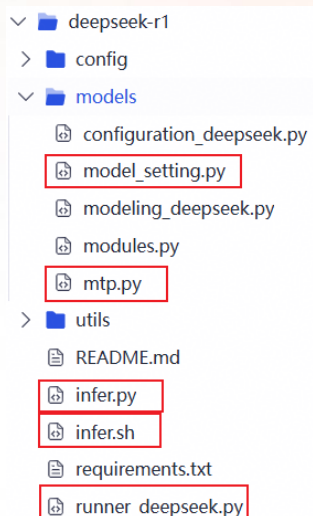
适配新模型：简化开发

重复开发 → 工程复用

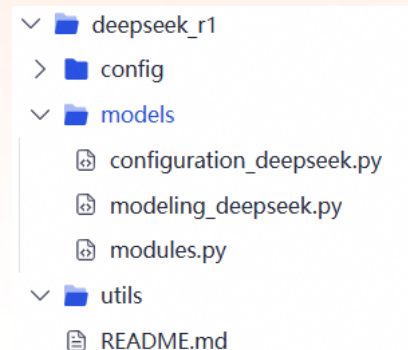
升级后，简化了模型适配工作，只需：

- 实现模型结构和 forward()。
- 声明 KV Cache 需求。
- 在 support_models.py 注册模型。

提供对应的文档 / 适配SKILL，可利用 Agent 沿着迁移清单逐项改造，降低使用门槛与适配成本。



减少代码量约 1k



● 调试

precision-debug

精度问题

runtime-debug

运行时错误

● 编排

optimize

端到端优化编排

sota-approach

候选发现 + Plan 自循环

● 并行

parallel-analysis

TP / EP / DP 选型

parallel-impl

并行切分实施

● 性能采集

profiling

采集 NPU prof

perf-breakdown

拆解 · 找抖动 layer

● 单点优化

fusion

kvcache

graph-mode

multi-stream

prefetch

quantization

统一模型使用，加速模型适配

适配新模型：一键测评

一键测试模型精度

用与离线推理相同的执行脚本，更改配置参数即可拉起在线服务：

```
bash executor/scripts/infer.sh --model <model> \  
--mode online \  
--pd-role prefill|decode
```

拉起Prefill端与Decode端服务后，调用evalscope测评工具：

```
evalscope eval \  
--model default \  
--api-url http://localhost:8000/v1 \  
--datasets gsm8k
```

```
2026-06-17 13:11:39 - evalscope - INFO: Evaluating[gsm8k] 27% 107/400 [Elapsed: 01:00 < Remaining: 02:26, 2.00it/s]  
2026-06-17 13:12:40 - evalscope - INFO: Evaluating[gsm8k] 55% 219/400 [Elapsed: 02:00 < Remaining: 01:28, 2.05it/s]  
2026-06-17 13:13:40 - evalscope - INFO: Evaluating[gsm8k] 81% 324/400 [Elapsed: 03:00 < Remaining: 00:48, 1.57it/s]  
2026-06-17 13:14:20 - evalscope - INFO: Evaluating[gsm8k] 100% 400/400 [Elapsed: 03:40 < Remaining: 00:00, 1.48it/s]  
Evaluating[gsm8k]: 100%  
2026-06-17 13:14:20 - evalscope - INFO: Unified pool finished for gsm8k.  
2026-06-17 13:14:20 - evalscope - INFO: Aggregating scores for subset: main  
2026-06-17 13:14:20 - evalscope - INFO: Generating report...  
2026-06-17 13:14:20 - evalscope - INFO:  
gsm8k report table:  


| Model   | Dataset | Metric   | Subset | Num | Score | Cat.0   |
|---------|---------|----------|--------|-----|-------|---------|
| default | gsm8k   | mean_acc | main   | 400 | 0.975 | default |

  
2026-06-17 13:14:20 - evalscope - INFO: Skipping report analysis ('analysis_report=False').  
2026-06-17 13:14:20 - evalscope - INFO: Dump report to: ./outputs/20260617_131039/reports/default/gsm8k.json  
2026-06-17 13:14:20 - evalscope - INFO:  
gsm8k perf table:  


| Model   | Dataset | Num | Avg Lat<br>(s) | Avg TTF1<br>(ms) | Avg TPOT<br>(ms) | Avg Thpt<br>(tok/s) | Avg In<br>Tok | Avg Out<br>Tok |
|---------|---------|-----|----------------|------------------|------------------|---------------------|---------------|----------------|
| default | gsm8k   | 400 | 8.1115         | -                | -                | 16.82               | 581.947       | 136.47         |

  
2026-06-17 13:14:20 - evalscope - INFO: Benchmark gsm8k evaluation finished.  
2026-06-17 13:14:20 - evalscope - INFO: Running[eval] 100% 1/1 [Elapsed: 03:41 < Remaining: 00:00, 221.37s/benchmark]  
2026-06-17 13:14:20 - evalscope - INFO: Overall report table:  


| Model   | Dataset | Metric   | Subset | Num | Score | Cat.0   |
|---------|---------|----------|--------|-----|-------|---------|
| default | gsm8k   | mean_acc | main   | 400 | 0.975 | default |

  
2026-06-17 13:14:20 - evalscope - INFO: Overall perf table:  


| Model   | Dataset | Num | Avg Lat<br>(s) | Avg TTF1<br>(ms) | Avg TPOT<br>(ms) | Avg Thpt<br>(tok/s) | Avg In<br>Tok | Avg Out<br>Tok |
|---------|---------|-----|----------------|------------------|------------------|---------------------|---------------|----------------|
| default | gsm8k   | 400 | 8.1115         | -                | -                | 16.82               | 581.947       | 136.47         |

  
2026-06-17 13:14:21 - evalscope - INFO: HTML report generated: ./outputs/20260617_131039/reports/report.html  
2026-06-17 13:14:21 - evalscope - INFO: Finished evaluation for default on ['gsm8k']  
2026-06-17 13:14:21 - evalscope - INFO: Output directory: ./outputs/20260617_131039  
[mwen] root@deserver-bos-e0117616-00041:~#
```


统一模型使用，加速模型适配

适配新模型：一键测评

```
[root@devserver-hps-e0117616-00041 scripts]# bash infer.sh
===== launch online inference (model=gpt_oss, role=prefill)
No Python process running infer.py or server.py was detected.
world_size is: 1
server_num is: 1
LOCAL_HOST=7.150.7.109, IPs[0]=7.150.7.109
Node Rank : 0
VC_TASK_INDEX >>> 0
[INFO] total ranks is 1, and use 1 ranks in actual!
=====
[root@devserver-hps-e0117616-00041 scripts]#
```

```
Processes: 7
Memory used: 1.3%
Swap used: 0.0%
Usage On: 79%
Users online: 0
```

```
[root@devserver-hps-e0117616-00041 data]# clear
[root@devserver-hps-e0117616-00041 data]# clear
[root@devserver-hps-e0117616-00041 data]# curl -s -X POST http://localhost:8000/v1/chat/completions -H 'Content-Type: applic
{"id":"chatcmpl-0c61a52e","object":"chat.completion","created":1782461746,"model":"default","choices":[{"index":0,"message":
age|The user asks: \"where is Beijing? Please write 30 words to show me.\" They want a 30-word description of Beijing's loc
nt. We need to mention that Beijing is in northern China, capital, etc. Let's craft: \"Beijing lies in northern China, about
edge of the North China Plain, serving as the nation's capital\"}, {"finish_reason":"length"}], "usage":{"prompt_tokens":80,"co
-hps-e0117616-00041 data]#
[root@devserver-hps-e0117616-00041 data]#
[root@devserver-hps-e0117616-00041 data]#
[root@devserver-hps-e0117616-00041 data]#
[root@devserver-hps-e0117616-00041 data]# curl -s -X POST http://localhost:8000/v1/chat/completions -H 'Content-Type: applic
{"id":"chatcmpl-8f0ba09c","object":"chat.completion","created":1782461761,"model":"default","choices":[{"index":0,"message":
age|The user asks: \"where is Paris? Please write 30 words to show me.\" They want a 30-word description of where Paris is.
nt. We need to produce exactly 30 words. Let's craft: \"Paris lies in northern France, on the Seine River, roughly 200 miles
r art, culture, and historic landmarks.\" Count words.\n\nCount\"}, {"finish_reason":"length"}], "usage":{"prompt_tokens":80,"co
[root@devserver-hps-e0117616-00041 data]#
```

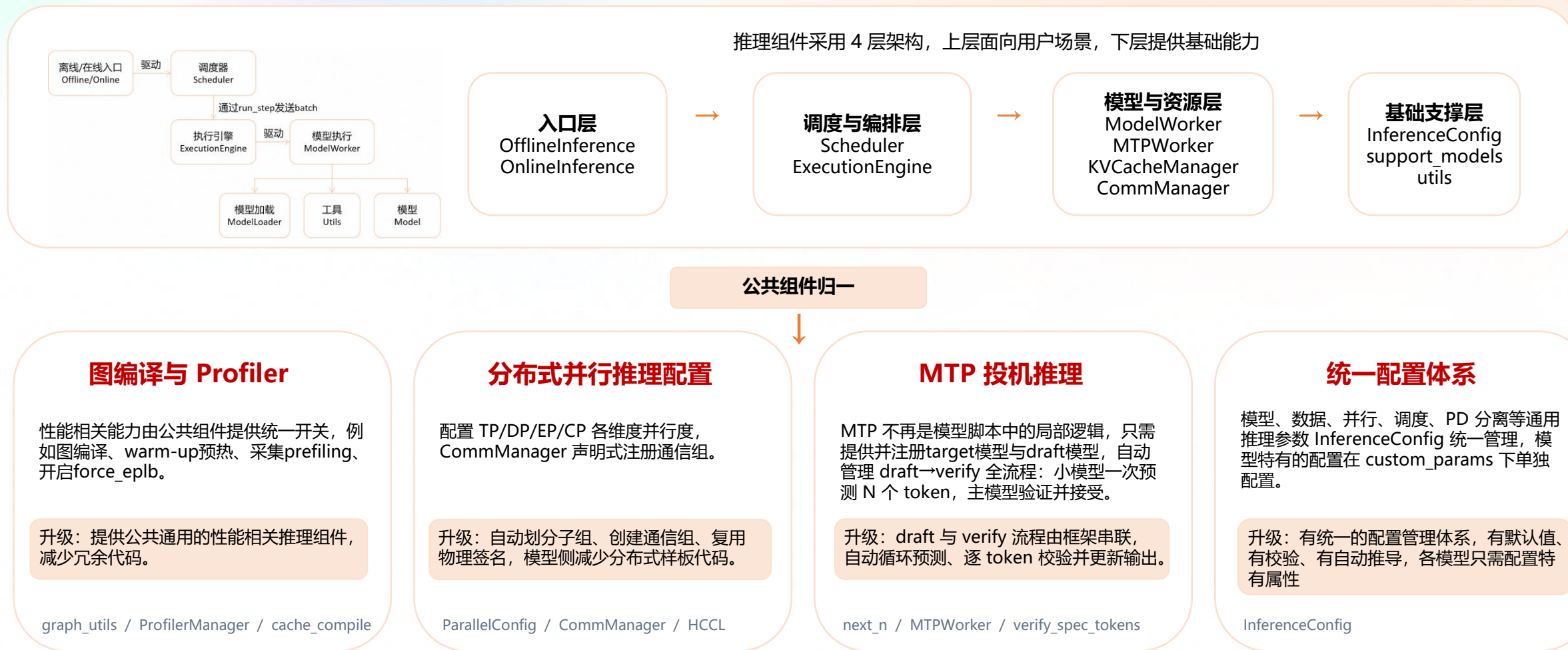
```
[root@devserver-hps-e0117616-00041 scripts]# bash infer.sh
===== launch online inference (model=gpt_oss, role=decode)
No Python process running infer.py or server.py was detected.
world_size is: 1
server_num is: 1
LOCAL_HOST=7.150.0.13, IPs[0]=7.150.0.13
Node Rank : 0
VC_TASK_INDEX >>> 0
[INFO] total ranks is 1, and use 1 ranks in actual!
=====
[root@devserver-hps-e0117616-00041 scripts]#
```

```
top - 16:23:22 up 2 days, 20:03, 12 users, load average: 19.23, 42.63, 45.43
Tasks: 2884 total, 1 running, 2883 sleeping, 0 stopped, 0 zombie
%cpu: 0.3 us, 0.3 sy, 0.0 ni, 99.4 id, 0.0 wa, 0.0 st, 0.0 sr, 0.0 ht
Mem Mem : 2634796 total, 665582 free, 16598.6 used, 1430885 buff/cache
Mem Swap: 0.0 total, 0.0 free, 0.0 used, 2634564 avail Mem
```

PID	USER	PR	NI	VSZ	RES	SHR	%CPU	%MEM	TIME+	COMMAND
321222	root	20	0	2114212	2400	15480	0.0	0.0	0:01:43.20	python3
296574	root	20	0	1852928	49144	12920	0.0	0.0	0:01:31.73	teleprobe
2645	root	20	0	82612	5260	1812	0.0	0.0	0:00:46.84	irgbalance
3186156	root	20	0	19.0p	849380	43140	0.0	0.0	10:50.66	node
846605	root	20	0	28184	6664	2912	0.0	0.0	0:02:23	top
2617	root	20	0	1316084	26144	6004	0.0	0.0	2:41.06	unigaint
13	root	20	0	0	0	0	0.0	0.0	11:25.30	rcu_sched
966	root	20	0	0	0	0	0.0	0.0	0:01.60	ksoftirqd/180
24708	root	-2	0	0	0	0	0.0	0.0	10:50.20	devd_rc_task
25324	root	-2	0	0	0	0	0.0	0.0	11:54.15	devd_rc_task
28054	root	-2	0	0	0	0	0.0	0.0	11:08.71	devd_rc_task
27724	root	-2	0	0	0	0	0.0	0.0	10:39.42	devd_rc_task
472284	root	20	0	0	0	0	0.0	0.0	0:00.41	kworker/73:0-mm_percpu_wq
503940	root	20	0	0	0	0	0.0	0.0	0:00.28	kworker/42:2-events
517601	root	20	0	0	0	0	0.0	0.0	0:00.13	kworker/320:0-events
586240	root	20	0	0	0	0	0.0	0.0	0:00.09	kworker/251:0-mm_percpu_wq
632281	root	20	0	0	0	0	0.0	0.0	0:01.71	kworker/7:0-events
670931	root	20	0	0	0	0	0.0	0.0	0:00.11	kworker/156:0-pcie_msg_workqueue
672644	root	20	0	0	0	0	0.0	0.0	0:00.05	kworker/187:1-mm_percpu_wq
676598	root	20	0	22384	5312	2880	0.0	0.0	0:04.20	shd
761123	root	20	0	1238660	15168	8008	0.0	0.0	0:00.06	containerd-shim
765519	root	20	0	0	0	0	0.0	0.0	0:00.02	kworker/24:4-events
815095	root	20	0	0	0	0	0.0	0.0	0:00.02	kworker/187:2-events
1	root	20	0	177452	11260	7362	0.0	0.0	0:07.81	systemd
2	root	20	0	0	0	0	0.0	0.0	1:17.13	htlthread
3	root	0	-20	0	0	0	0.0	0.0	0:00.00	rcu_gp
4	root	0	-20	0	0	0	0.0	0.0	0:00.00	rcu_par_gp
6	root	0	-20	0	0	0	0.0	0.0	0:00.00	kworker/25:0-events

推理组件实现细节

公共能力由框架统一维护，新模型迁移聚焦模型结构与性能优化，不再重复适配输入组织、图模式、并行通信、MTP 和配置校验

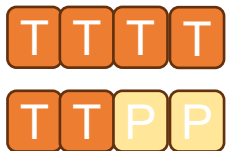


推理组件实现细节

新增组件：Packed Sequence 统一输入，KV Cache 由框架统一管理

Packed Sequence: 统一输入组织方式

在 prefill 与 decode 时, input_ids 按 [TotalTokens] 拼接。



$[T, N, D] = [6, 4, 64]$
cu_seqlens(序列边界)

$[B, S, N, D] = [2, 4, 4, 64]$

有效计算: $6/8=75\%$

$[0, 4, 6]$

有效计算: $6/6=100\%$

升级: Packed Sequence 通过紧凑组织**真实 tokens**, 减少prefill阶段的无效计算, 同时, ForwardMetaData 可以给模型内部提供复杂的位置和请求边界管理等信息。

KVCache 管理组件: PagedAttention

ModelWorker.get_cache_info()
模型自描述 KV Cache 需求

ExecutionEngine.init_cache_manager()
按 attn_type 分组创建 Manager

KVCacheManager
请求级顶层协调者

FullAttentionManager
全注意力, 永不丢弃 token

SlidingWindowManager
窗口外 block 自动回收

SingleTypeKVCacheManager
管理不同 attention 类型

ExecutionEngine
自动计算block_table与slot_mapping

Attention
调用并更新cache

BlockPool 块池
O(1) 分配/回收, null_block 占位

升级: 用户只需在模型内描述KV Cache的需求, 例如cache_name, attn_type, dtype, shape 等, 即可通过 KV Cache管理组件自动申请物理Tensor块、维护不同请求的block申请与释放、计算block_table和slot_mapping等操作。

推理组件实现细节

新增在线推理功能，支持数据集精度测评判断模型精度

离线推理：稳定、可复现、适合性能定位

offline 是框架的核心使用场景，适合固定输入、固定 batch、固定配置下进行性能验证。

适用场景：

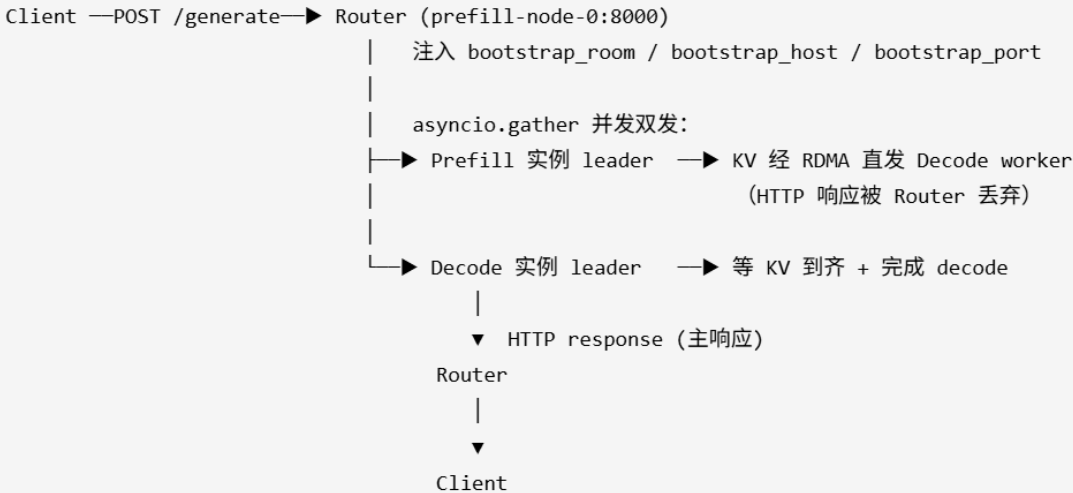
- 单模型性能 Benchmark。
- 图编译、MTP、KV Cache、并行策略的定点调试。

在线推理：服务化入口，兼容评测工具

online 提供简易 Client-Server 能力，便于用 HTTP 接口连接 **evalscope** 评测工具。

适用场景：

- 对量化、算子替换等优化前后结果进行端到端**数据集精度测评**，辅助判断精度影响。



PD 分离：Prefill 与 Decode 分角色部署

在线模式支持 Prefill-Decode 分离，使两类计算按不同资源策略部署。

组件	作用
Router	为同一请求选择 prefill / decode 实例，并注入 bootstrap_room
Bootstrap Server	提供 rank table 注册、查询和请求路由信息
PrefillDisaggScheduler	prefill 完成后触发 KV 传输
DecodeDisaggScheduler	管理 prealloc、transfer、running 队列
KVTransferManager	控制面走 ZMQ，数据面走 RDMA / HCCL 后端



说明：当前在线能力优先保证可用性和测评便利，不以完整服务框架的吞吐优化为首要目标。

总结与未来计划

开箱即用：易用性、精度验证、适配效率、工程复用同步提升。

推理组件持续优化

在当前公共组件基础上持续扩展模型覆盖范围，推动 LLM 样例全面接入；
针对多模态模型特性也将在当前组件能力上继续开发，并逐步接入统一推理流程。

文档建设

完善原子特性文档建设，对每个独立能力说明原理、适用场景、配置方式和优化效果。

将分散在仓库中的文档材料迁移至独立 Blog，形成面向用户的统一阅读入口；

欢迎体验：<https://gitcode.com/cann/cann-recipes-infer>

CANN RECIPES



Cann-recipes-infer交流群



讨论区



Cann-recipes-infer新手村任务

Infer

定位：面向 NPU 推理场景的模型实践与执行框架。
提供：模型样例、统一启动脚本、公共推理组件、性能优化实践文档。
案例：DeepSeek-v4系列推理优化样例、HunyuanVideo推理优化样例
<https://gitcode.com/cann/cann-recipes-infer>



Train

定位：面向昇腾CANN平台的大模型训练样例集合。
提供：LLM/多模态场景预训练与后训练（SFT、RL、Agentic RL等）高性能NPU训练优化样例、性能调优材料。
案例：DeepSeek-V4-Flash/Pro 续训练样例、Qwen3-235B/Qwen3-32B RL训练样例
<https://gitcode.com/cann/cann-recipes-train>



AMCT

定位：昇腾 NPU 原生的大模型量化压缩工具仓。
提供：主流 LLM 网络量化、PTQ 算法支持。
案例：Qwen3.6-MoE量化样例、DeepSeek-v4端到端后训练量化样例
<https://gitcode.com/cann/amct>



Embodied-ai

定位：基于CANN平台的具身智能任务的优化样例。
提供：端到端任务样例、感知/决策链路、场景化验证和部署参考。
案例：Pi0通用机器人控制训推样例、VGGT三维视觉推理样例
<https://gitcode.com/cann/cann-recipes-embodied-ai>



THANKS

